

Routing

COMPLETE LEARNING GUIDE

What You'll Learn

Single-page applications load a single HTML document and then use JavaScript to swap what's displayed as the user navigates, rather than requesting a fresh page from the server on every link click — routing is the layer that makes this feel like ordinary multi-page browsing while staying entirely client-side. This submodule covers React Router (the dominant routing library in the React ecosystem) in depth: declaring routes, matching URL parameters, nested routes with shared layouts via Outlet, programmatic navigation, and protecting routes behind authentication checks — with the equivalent concepts in Angular's built-in Router and Vue Router called out for comparison. It also covers route-based code splitting, which is one of the highest-leverage, easiest performance wins available in any client-rendered SPA.

1. Why Client-Side Routing Exists

Traditional multi-page sites request a brand-new HTML document from the server on every navigation, which resets all JavaScript state and re-downloads shared assets. A client-side router instead intercepts navigation, uses the browser's History API (`pushState/replaceState`) to update the URL without a full page reload, and swaps the rendered component tree to match — all state that should persist (a global cart, an auth session already in memory) survives navigation, and only the data actually needed for the new view has to be fetched.

2. Basic Setup with React Router

React Router wraps the app in a `BrowserRouter` (which hooks into the History API), then declares a `Routes` container holding individual `Route` elements, each mapping a URL path to a component to render. Routes are matched by path, and the first `Route` whose path matches the current URL renders its element — React Router v6+ matches routes more predictably than earlier versions by always picking the best (most specific) match rather than the first one written in source order.

JSX

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/courses" element={<CourseList />} />
        <Route path="/courses/:courseId" element={<CourseDetail />} />
        <Route path="*" element={<NotFound />} /> { /* catch-all 404 */}
      </Routes>
    </BrowserRouter>
  );
}
```

3. Route Parameters and Nested Routes

A path segment prefixed with a colon (:courseId) captures that portion of the URL as a route parameter, read inside the matched component via the `useParams()` hook — note that all route params arrive as strings, even ones that look numeric, so parsing (`Number(id)`) is often needed before using them. Nested routes let a parent route render shared layout (a sidebar, a tab bar) once, with an `<Outlet />` placeholder marking where the matched child route's content should render — this avoids re-rendering shared chrome on every navigation within that section.

JSX

```
function CourseDetail() {
  const { courseId } = useParams(); // always a string, e.g. "42"
  const [course, setCourse] = useState(null);

  useEffect(() => {
    fetchCourse(courseId).then(setCourse);
  }, [courseId]); // re-fetch whenever the route param changes

  return course ? <h1>{course.title}</h1> : <p>Loading...</p>;
}

// Nested routes sharing a layout via <Outlet />
<Route path="/courses/:courseId" element={<CourseLayout />}>
  <Route index element={<CourseOverview />} />
  <Route path="lessons" element={<CourseLessons />} />
</Route>
```

4. Programmatic Navigation and Links

Regular navigation should always use the `<Link to="/path">` component instead of a plain `<a href="/path">`, because `Link` intercepts the click and performs client-side navigation through the router, while a plain `<a>` tag triggers a full page reload that defeats the entire purpose of an SPA. For navigation that needs to happen as a result of logic rather than a direct click (redirecting after a successful form submission, for example), the `useNavigate()` hook

returns a function that can push a new route programmatically.

JSX

```
import { Link, useNavigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  async function handleSubmit(credentials) {
    await login(credentials);
    navigate('/dashboard'); // programmatic redirect after successful login
  }

  return (
    <>
      <form onSubmit={handleSubmit}>{/* ... */}</form>
      <Link to="/signup">Need an account? Sign up</Link>
    </>
  );
}
```

5. Protected Routes

Restricting a route to authenticated users is typically implemented as a wrapper component that checks auth state and either renders its children or redirects to a login page using the Navigate component. This pattern composes cleanly with nested routes — wrap a whole subtree of protected routes in a single RequireAuth layout route instead of repeating the check inside every individual protected page.

JSX

```
function RequireAuth({ children }) {
  const { isAuthenticated } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    // Preserve where the user was headed so we can send them back after login
    return <Navigate to="/login" state={{ from: location }} replace />;
  }
  return children;
}

<Route path="/dashboard" element={<RequireAuth><Dashboard /></RequireAuth>} />
```

6. Route-Based Code Splitting

Shipping the JavaScript for every single route in one bundle means users pay the download cost for pages they may never visit. React's lazy() combined with Suspense lets a route's component code be fetched only when that route is actually navigated to, splitting the bundle automatically along route boundaries — one of the highest-value, lowest-effort performance techniques available in any React SPA of meaningful size.

JSX

```
import { lazy, Suspense } from 'react';

const AdminPanel = lazy(() => import('./AdminPanel'));

<Route
  path="/admin"
  element={
    <Suspense fallback={<p>Loading...</p>}>
      <AdminPanel />
    </Suspense>
  }
/>
```

7. Routing in Angular and Vue

Angular's built-in RouterModule configures routes as an array of {path, component} objects (or, in modern Angular, functional route configs with loadComponent for lazy loading), and templates use <router-outlet> as the equivalent of React Router's <Outlet />, with routerLink replacing href for internal navigation. Vue Router configures routes similarly as an array of {path, component} objects passed to createRouter(), uses <router-view> as its outlet, and <router-link to="..."> for navigation. All three frameworks converge on the identical mental model — a route table mapping paths to components, an outlet marking where matched content renders, and a special link component that avoids full page reloads.

Common Pitfalls

- Using a plain instead of <Link>/<router-link>, triggering an unwanted full page reload.
- Forgetting that route params are always strings, then comparing/using them as numbers without converting first.
- Not including a catch-all wildcard route, leaving users with a blank page instead of a proper 404 view on an unmatched URL.
- Re-implementing an auth check inside every individual protected page instead of a single reusable RequireAuth wrapper route.
- Shipping every route's code in one bundle instead of lazy-loading route components, inflating the initial page load.
- Forgetting the dependency array on a data-fetching useEffect keyed on a route param, so navigating between two similar routes doesn't refetch.

Key Takeaways

- Client-side routing swaps rendered components via the History API instead of requesting a fresh document — the URL changes, but the page never fully reloads.

- Always navigate internally with the router's link component, never a plain `<a href>`, to avoid a full page reload.
- Route params always arrive as strings, regardless of what they look like.
- Nested routes with an Outlet let shared layout render once instead of re-rendering on every navigation within a section.
- Lazy-loading route components via code splitting is one of the easiest, highest-impact performance wins in any sizeable SPA.

Learn More

React Router — Official Documentation

<https://reactrouter.com/>

Vue Router — Official Documentation

<https://router.vuejs.org/>

Angular — Routing & Navigation

<https://angular.dev/guide/routing>